Lecture 6
Remotes and GitHub: Sharing Your Work

`clone`, `push`, `fetch`, `pull`, SSH keys, and your first GitHub repository

Learning goals

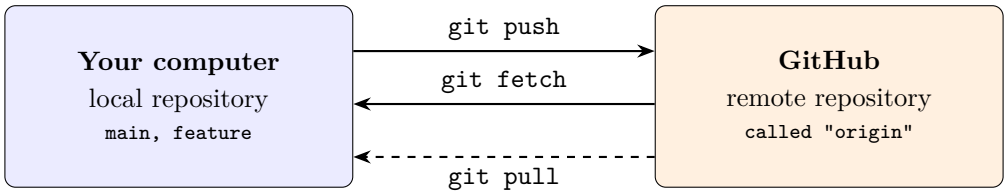
- Explain what a *remote* is and what `origin` and `origin/main` mean.
- Set up GitHub safely: account, private email address, and an SSH key.
- Put a local repository on GitHub, and copy a GitHub repository to your computer.
- Use `git push`, `git fetch` and `git pull`, and understand "ahead", "behind" and "diverged".
- Fix a rejected push.

1 Why a remote?

So far everything lived on one computer. That is risky (a lost laptop loses the project) and lonely (nobody else can see or add to it). A **remote** is simply *another copy of the repository, stored somewhere else*, usually on a server that several people can reach. **GitHub** is the most popular place to keep one.

(For the first part of this lecture and Homework Part A, the "server" will be just a folder on your own computer, a so-called *bare repository*: a repository without a working folder, which is what a server holds. Git treats a folder path exactly like a GitHub address, so everything works the same. The GitHub account, SSH key and publishing steps come after the core ideas and are needed only for Part B.)

Remember Lecture 0: Git is *distributed*. Your computer holds a complete repository, GitHub holds another complete repository, and a friend's computer holds a third. They are equals in structure; they stay in step because people *send* commits between them.



`git clone` copies a remote to your computer once, at the start

Figure 1: Two full copies of a repository. `push` sends your new commits up; `fetch` brings new commits down; `pull` (dashed) is a fetch followed by a merge.

2 Copying a repository: `git clone`

To get an existing repository, for example a friend's, you *clone* it. On GitHub the green **Code** button shows the address.

```
$ git clone git@github.com:friend/project.git
Cloning into 'project'...
$ cd project
$ git remote -v
origin  git@github.com:friend/project.git (fetch)
```

```
origin  git@github.com:friend/project.git (push)
$ git log --oneline --decorate
4c2511d (HEAD -> main, origin/main, origin/HEAD) Add notes
3db1fe7 Add README
```

(The address may be a GitHub address, as here, or just a folder path such as `hw6_server.git`, as in the homework.) One command created a new folder with the *whole history*, set up the remote called `origin`, and checked out the main branch. (`git clone <address> mydir` picks another folder name.) You do not run `git init` first.

3 Remote-tracking branches: `origin/main`

Look at the log above: it shows both `main` and `origin/main`. The notation is `<remote name>/<branch name>`, and they are different things. (`origin/HEAD` is the remote's default branch; ignore it.)

- `main` is *your* branch. It moves when you commit.
- `origin/main` is a **remote-tracking branch**: your computer's *memory* of where `main` was on `origin` the last time you talked to it. You never commit on it; Git moves it only when you push, fetch or pull.

Comparing the two tells Git (and you) whether you are ahead of, behind, or in step with the remote:

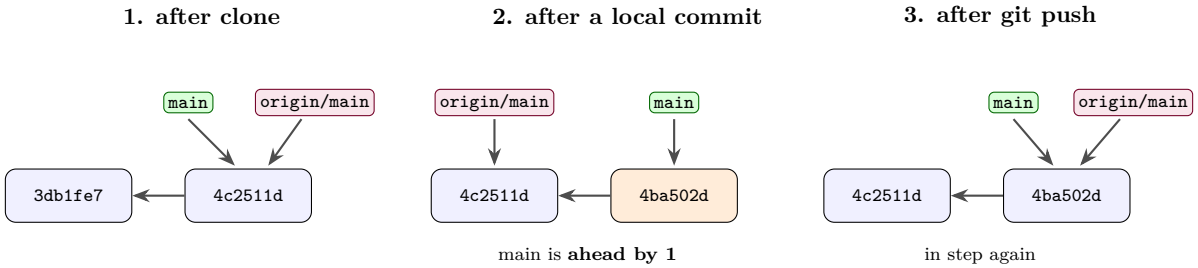


Figure 2: `origin/main` (purple) stays where the remote was last seen. Committing moves only `main`; pushing moves both.

`git status` reports the comparison in words:

```
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

Key idea

`origin/main` is only as fresh as your last contact. `git status` never phones GitHub: if a friend has pushed since, it will still say "up to date" until you `fetch`.

4 Getting others' work: `fetch` and `pull`

A friend pushes a commit. Your computer does not know yet. `git fetch` downloads new commits and moves `origin/main`, but *touches neither your branch nor your files*, so it is always safe:

```
$ git fetch
From github.com:friend/project
   4c2511d..322927b  main       -> origin/main
$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
$ git log --oneline main..origin/main        # what is new on the remote?
322927b Friend update
```

"Behind, and can be fast-forwarded" is the easy case from Lecture 5: your branch has nothing new, so it just slides forward to catch up. `git pull` does exactly a fetch followed by a merge, here a fast-forward:

```
$ git pull
Updating 4c2511d..322927b
Fast-forward
 friend.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 friend.txt
```

4.1 When both sides have new commits

The harder case: while you made a commit locally, your friend pushed one. Now the two copies have *diverged*, exactly like the two branches of Lecture 5. Git protects the shared copy and rejects your push:

```
$ git push
To github.com:friend/project.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'github.com:friend/project.git'
hint: Updates were rejected because the remote contains work that you do
hint: not have locally. This is usually caused by another repository pushing
hint: to the same ref. You may want to first integrate the remote changes
hint: (e.g., 'git pull ...') before pushing again.
```

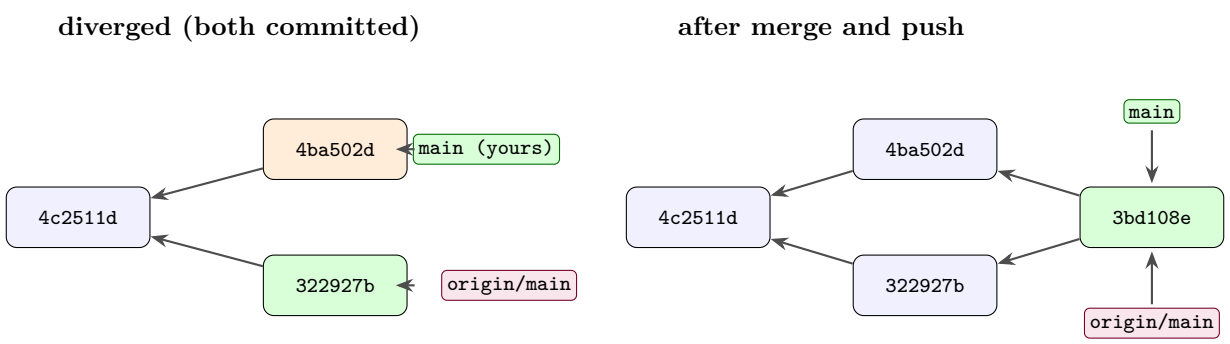


Figure 3: Left: your `main` and `origin/main` point at different commits. Right: a merge commit joins them; after the push both labels sit on it.

The cure is the merge you learned in Lecture 5: first bring your friend's commit in, then push. Git 2.33 and newer, however, insist that you *choose* how `pull` should combine two diverged branches (if you see no such message, your Git is already configured, so carry on):

```
$ git pull
From github.com:friend/project
   4c2511d..322927b  main       -> origin/main
hint: You have divergent branches and need to specify how to reconcile them.
hint: You can do so by running one of the following commands sometime before
hint: your next pull:
hint:   git config pull.rebase false  # merge
hint:   git config pull.rebase true   # rebase
hint:   git config pull.ff only       # fast-forward only
fatal: Need to specify how to reconcile divergent branches.
```

(Some hint lines are left out.) Notice that the fetch part already worked: `origin/main` moved. Only the combining step is waiting for your decision. We choose **merge**, which you know, and say so once for all repositories with `--global` (rebase is Lecture 8; the homework sets it for one repository only):

```
$ git config --global pull.rebase false
$ git pull --no-edit              # fetch + merge origin/main
Merge made by the 'ort' strategy.
 friend.txt | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 friend.txt
$ git push
To github.com:friend/project.git
   322927b..3bd108e  main -> main
```

The same thing, step by step, is `git fetch` followed by `git merge origin/main`. If the merge ever conflicts, resolve it exactly as in Lecture 5, then commit and push.

Quick fact

Never use `git push --force` to get around a rejection. It overwrites the remote with your version and *deletes your friend's commits* from it. Beginners should treat it as forbidden until Lecture 8.

5 Branches and the remote

A new branch exists only on your computer until you push it. Use `-u` the first time. It sets the branch's *upstream*: it pairs your local branch with the remote branch of the same name ("set up to track" in the output below), so afterwards a plain `git push` or `git pull` knows where to go.

```
$ git switch -c feature
$ git push -u origin feature
 * [new branch]      feature -> feature
branch 'feature' set up to track 'origin/feature'.
$ git branch -vv
* feature d74b2c7 [origin/feature] Add feature file
  main    3bd108e [origin/main] Merge branch 'main' of github.com:friend/project
$ git branch -r
  origin/HEAD -> origin/main
  origin/feature
  origin/main
```

`git branch -vv` shows which remote branch each local branch tracks; `git branch -r` lists the remote-tracking branches. To delete a branch on the remote once it is merged: `git push origin --delete feature`. (Git removes your own `origin/feature` label at once. If someone else deletes a branch on GitHub, for example with the website's "Delete branch" button, `git fetch --prune` removes the stale label on your side.)

6 Setting up GitHub

Do these once. They take about ten minutes. **Homework Part A does not need them; they are needed for Part B**, so you can also do Part A first.

6.1 Account and your private email address

Create a free account at `https://github.com` and turn on two-factor authentication (Settings, "Password and authentication"). Choose your user name carefully: it appears in every repository address.

In Lecture 1 you put your email address into `git config`, and in Lecture 5 you saw that every commit carries it. Commits pushed to GitHub show that address *publicly*. GitHub offers a private "no-reply" address that works instead:

1. In GitHub **Settings** → **Emails**, tick *Keep my email addresses private* and, below it, *Block command line pushes that expose my email*.
2. The same page shows your no-reply address: a number, a plus sign, your user name, and `@users.noreply.github.com`. Copy it from there rather than typing it.
3. Tell Git to use it for future commits:

```
$ git config --global user.email "12345678+yourname@users.noreply.github.com"
```

Commits you have *already* made (such as those of Lectures 1–5) keep the old address; the setting changes only new commits. Rewriting old history to change them is possible but out of scope here, so do not worry about it. Check the setting any time with `git config --global user.email`.

6.2 Authentication: proving who you are

To `push` you must prove that the repository is yours. GitHub no longer accepts your account password for this. There are two ways:

- **HTTPS with a token.** Repository addresses start with `https://`, and you paste a long, secret *personal access token* where a password would go.
- **SSH with a key** (recommended). Repository addresses look like `git@github.com:yourname/repo.git`. You create a pair of key files once; afterwards you do not paste a token on every push.

An **SSH key** is a pair of files, like a padlock and its key. The *public* half is the padlock: you may give it to anyone, and you put it on GitHub. The *private* half is the key that opens it: it never leaves your computer.

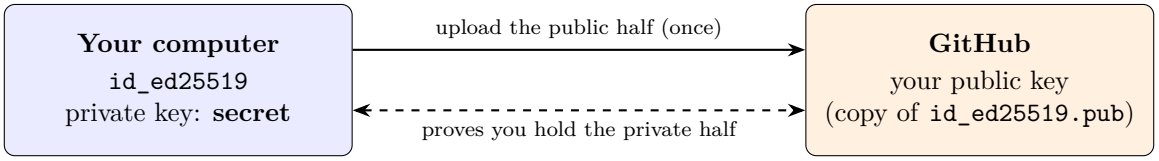


Figure 4: A key pair. Only the public half is ever uploaded.

Create the key (in a terminal; in Git Bash on Windows). Use your own email address as the label, and a passphrase when asked:

```
$ ssh-keygen -t ed25519 -C "you@example.com"
Enter file in which to save the key (/Users/you/.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
```

At the first question just press Enter. At the passphrase question type a passphrase (a password for the key file); *no characters appear while you type*, which is normal. Type it again to confirm. (If it says the key already exists, answer `n` and reuse the existing key.)

This creates two files in the hidden folder `~/.ssh/`: `id_ed25519` (private) and `id_ed25519.pub` (public). Print the public one and copy the whole line:

```
$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3Nza...(long)... you@example.com
```

On GitHub open **Settings** → **SSH and GPG keys** → **New SSH key**, give it a title (for example "my laptop"), paste the line, and save. Then test the connection:

```
$ ssh -T git@github.com
Hi yourname! You've successfully authenticated, but GitHub does not provide shell
    access.
```

That message is success: GitHub knows you, and (as it says) does not give you a command line on its servers. The first time, SSH asks whether to trust the server; compare the fingerprint it prints with the one on GitHub's documentation page "GitHub's SSH key fingerprints", then answer `yes`.

If you set a passphrase, your computer may ask for it on every push. Let it remember the passphrase for this session with `ssh-add ~/.ssh/id_ed25519` (on macOS: `ssh-add --apple-use-keychain ~/.ssh/id_ed25519`). On Windows, Git Bash shows your home folder as `/c/Users/yourname`, and you may first need `eval "$(ssh-agent -s)"` to start the agent.

Pitfall

If the test fails. `Permission denied (publickey)` means GitHub does not know your key: re-copy the *whole* line of the `.pub` file into GitHub (not the private file), and check that you added it to your own account. `Could not resolve hostname` means no internet connection.

Pitfall

Never share, e-mail or upload `id_ed25519` (the private key; the file ending in `.pub` is the safe one), and never commit a `.ssh` folder. If you suspect it leaked, delete the key on GitHub and create a new pair. The passphrase protects the file if your laptop is stolen.

Quick fact

If you prefer HTTPS: create a *classic* token under Settings → Developer settings → Personal access tokens, tick only the scope *repo*, and paste it when Git asks for a password. In Part B copy the HTTPS address instead of the SSH one; the checker accepts both. Your operating system can remember it (the *credential helper*). Everything else in this lecture is identical: only the address style differs.

7 Putting a local repository on GitHub

You already have a local repository. To share it:

1. On GitHub click **New repository**. Give it a name and choose public or private.
2. **Do not** tick "Add a README", ".gitignore" or "license". Those would create a first commit on GitHub that your local history does not contain, and the two histories would not fit together. You want an *empty* repository.
3. GitHub now shows its quick-setup page with the repository's address. Copy the SSH address, and read the warning below before you copy any snippet.

Pitfall

If you ticked "Block command line pushes that expose my email" (Section 6) and your commits still carry your real address, the push is refused with `GH007: Your push would publish a private email address`. Set the no-reply address with `git config user.email` *before* making the commits you push (without `--global` it applies to one repository only).

Pitfall

That page offers two snippets, and the wrong one can do damage. The *first* ("create a new repository on the command line") starts with `echo "# name" >> README.md` and `git init`. It is for a *brand-new, empty folder*. Run inside an existing project, it appends a stray line to your README. You want the *second* ("push an existing repository"), which has three lines. One of them, `git branch -M main`, *renames the branch you are currently on* to `main`, so check with `git branch --show-current` that you are on the branch you mean. (`-M` also *overwrites* an existing branch of that name: run on `feature` while `main` exists, it replaces `main`.)

The three commands, run inside your existing repository:

```
$ git remote add origin git@github.com:yourname/hello-github.git
$ git remote -v
origin  git@github.com:yourname/hello-github.git (fetch)
origin  git@github.com:yourname/hello-github.git (push)
$ git push -u origin main
To github.com:yourname/hello-github.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.
```

(Real output also shows progress lines such as `Counting objects`, left out here.) Line by line:

- `git remote add origin <address>` gives the remote a short name. `origin` is the conventional name for "the place I got this from or share it with"; it is only a label.
- `git remote -v` lists your remotes. To change an address later: `git remote set-url origin <new-address>`.

- `git push -u origin main` sends the commits of the branch `main` to `origin`. The `-u` ("upstream") makes your `main` *remember* that it belongs with `origin/main`, so that afterwards a plain `git push` or `git pull` is enough.

8 What GitHub adds

The website shows your repository: files, the commit list, branches, and the `README.md` rendered as a front page. A README is written in **Markdown**, a plain-text format where `# Title` is a heading, `**bold**` is **bold**, `- item` starts a list, and backticks make `code`. You can also edit a file in the browser (the pencil icon) and "commit changes" there. That creates a *commit on GitHub*, which your local copy does not have until you `git pull`.

Pitfall

Everything you push is as visible as the repository: anyone can read a *public* repository, and history keeps every old version. Never push passwords, keys or tokens (Lecture 2), even in a file you later delete.

9 Homework 6

Homework 6: Sharing

Part A: practice with a "server" folder (no internet needed). The setup script creates `practice/hw6_server.git`, a bare repository that plays the role of GitHub. Cloning, pushing, fetching and pulling work with it exactly as with GitHub. From the course folder run:

```
$ bash codeHints/L06/setup_hw6.sh
$ cd practice
```

Do the tasks in this order, and *do not push before 6.4*. To start over (also if you ran `friend.sh` too early): `setup_hw6.sh --reset` from the course folder.

6.1 Clone

Clone the server into a folder called `hw6` (`git clone hw6_server.git hw6`) and go into it.

6.2 Look around

Run `git remote -v`, `git branch -a` and `git log --oneline --decorate`. In `practice/hw6_answers.txt` (next to the `hw6` folder; create it with `printf` or `echo` in Git Bash, not with a PowerShell redirect) write the line `remote=<the name of the remote>`.

6.3 Commit locally

Create `mine.txt` containing `my work` and commit it as `Add my work`. *Do not push yet, and do not run `friend.sh` yet.* Run `git status` and add the line `ahead=<number of commits ahead>` to the answers file.

6.4 Your friend pushes

Go back to the *course folder* (`cd ../..`) and run `bash codeHints/L06/friend.sh`. Then `cd practice/hw6` again and try `git push` and read the rejection. Set `git config pull.rebase false`, then run `git pull --no-edit`. Check `git log --oneline --graph`: there is now one merge commit.

6.5 Push

Run `git push`. Your `main` and the server's `main` must now be identical.

6.6 A branch
Create the branch `feature`, add a file `feature.txt`, commit it as `Add feature file`, and push it with `git push -u origin feature`.

6.7 Question
Add the line `no-merge=<command>`: the one Git command from this lecture that downloads new commits *without* changing your branch or your files.

Part B: the real thing (optional, but this is why you came). You need your GitHub account and the SSH key from this lecture.

6.8 Create and push
Do the account, privacy and SSH setup above first. On GitHub create a new *empty* repository called `hello-github` (no README). In `practice/` create the folder `hw6_github`, run `git init -b main` in it (`-b main` names the first branch, as in Lecture 1), add a `README.md` with a heading line such as `# Hello GitHub`, commit, add the SSH address as `origin`, and `git push -u origin main`.

6.9 Edit on GitHub, then pull
In the browser open `README.md`, click the pencil icon, change the text, and press *Commit changes*. Then run `git pull` locally and check that `git log` shows the new commit.

6.10 Look, then check
On github.com, find your commits in the commit list. Then run the checker.

Check yourself: from the course folder run `bash codeHints/L06/check_hw6.sh`. Part A needs no network. Part B is checked only if `practice/hw6_github` exists; if GitHub cannot be reached, that one check is skipped. All `PASS`, then compare with the solutions at the end of this lecture (or in `solutions/L06/`).

10 Summary

- A remote is another full copy of the repository; `origin` is the usual name for the one you share with. `origin/main` is your memory of where `main` was on it.
- Set up GitHub with a private no-reply email and an SSH key (public half uploaded, private half secret).
- `git remote add`, `git push -u origin main` publish a local repo (create it *empty* on GitHub); `git clone` copies one.
- `fetch` downloads safely; `pull` is fetch plus merge. If both sides have new commits, pull (merge) first, then push. Never force-push.

Further reading: *Pro Git*, Chapters 2.5 and 3.5; GitHub Docs, "Connecting to GitHub with SSH".

Solutions

Stop here if you have not tried the homework yet.
Attempt every task first, run the checker, and only then read on.

Solutions to Homework 6

Part A.

```
$ bash codeHints/L06/setup_hw6.sh
$ cd practice
$ git clone hw6_server.git hw6
$ cd hw6
$ git remote -v                      # origin ... (fetch), origin ... (push)
$ git branch -a
* main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
$ echo "my work" > mine.txt
$ git add mine.txt
$ git commit -m "Add my work"
$ git status                         # "ahead of 'origin/main' by 1 commit"
$ printf 'remote=origin\nahead=1\nno-merge=fetch\n' > ../hw6_answers.txt
$ cd ../..                           # back to the course folder
$ bash codeHints/L06/friend.sh
$ cd practice/hw6
$ git push                           # rejected (fetch first)
$ git config pull.rebase false
$ git pull --no-edit                 # fetch + merge: one merge commit
$ git log --oneline --graph
$ git push
$ git switch -c feature
$ echo "feature" > feature.txt
$ git add feature.txt
$ git commit -m "Add feature file"
$ git push -u origin feature
```

6.1–6.2. `git clone` created `hw6` with the full history and a remote named `origin` that points at the server folder. The remote's name is `origin` (Git chooses it for a clone), hence `remote=origin`.

6.3. Your branch has one commit that `origin/main` lacks, so `git status` says "ahead by 1 commit" and `ahead=1`. Nothing is on the server yet.

6.4. The friend's commit reached the server after you cloned, so the two copies diverged and `git push` is rejected with "fetch first". Plain `git pull` on a recent Git stops with "Need to specify how to reconcile divergent branches" (after already fetching); `git config pull.rebase false` chooses merge. The merge commit is the *one* merge commit the checker expects. (`git fetch` followed by `git merge origin/main` gives the same result.)

6.5. After the merge your `main` is ahead of `origin/main`, and a push is now a fast-forward for the server, so it is accepted.

6.6. `-u` records that local `feature` tracks `origin/feature` (the checker reads `branch.feature.remote` and `branch.feature.merge` from your repository).

6.7. `fetch`: it downloads and moves `origin/main` but changes neither your branches nor your files. `git pull` also merges.

Part B.

```
$ cd practice
$ mkdir hw6_github
$ cd hw6_github
$ git init -b main
$ git config user.email "ID+YOURNAME@users.noreply.github.com"   # if you use the no-
    reply address
$ echo "# Hello GitHub" > README.md
$ git add README.md
$ git commit -m "Add README"
$ git remote add origin git@github.com:YOURNAME/hello-github.git
$ git push -u origin main
        (now edit README.md on github.com, "Commit changes")
$ git pull
$ git log --oneline                  # two commits
```

Replace `YOURNAME` by your GitHub user name. After the browser edit, GitHub is one commit ahead of you, so `git pull` brings it in as a fast-forward and your local `main` again equals GitHub's.

Common mistakes and fixes.

- Running `friend.sh` before cloning, or pushing before 6.4. Then there is no divergence and no merge commit, so the check "exactly one merge commit" fails. Run `setup_hw6.sh --reset` and follow the order.
- `Permission denied (publickey)` when pushing to GitHub: the key is not added to your account, or the remote uses the wrong address style. Test with `ssh -T git@github.com`; check `git remote -v`.
- Creating the GitHub repository *with* a README, then pushing: rejected, because GitHub has a commit you lack. Either create it empty (best) or run `git pull --no-rebase origin main --allow-unrelated-histories`; both sides have a `README.md`, so expect an add/add conflict there: resolve it as in Lecture 5, commit, and push.
- Using `git push --force` to "fix" a rejection: not allowed here, it erases others' work.